

RAPPORT DE PROJET : MACHINE LEARNING

Modélisation et Prédiction de la Demande de Vélos en Libre-Service

TABLE DES MATIÈRES

1. Introduction et État de l'Art
2. Étape 1 : Analyse Exploratoire des Données
3. Étape 2 : Pré-processing et Feature Engineering
4. Étape 3 : Évaluation de base
5. Étape 4 : Tuning, Optimisation et Diagnostic de l'Overfitting
6. Étape 5 : Industrialisation et Sérialisation du Pipeline
7. Étape 6 : Architecture de Déploiement et Production
8. Conclusion

1. INTRODUCTION ET ÉTAT DE L'ART

Problématique Métier :

La gestion d'un réseau urbain de vélos en libre-service repose sur un défi logistique majeur : l'équilibrage des stations. Pour éviter qu'une station ne se retrouve saturée ou totalement vide, il est essentiel d'anticiper précisément la demande future. Ce projet utilise l'apprentissage automatique pour prédire le volume horaire des locations en fonction de facteurs temporels et environnementaux.

Définition Formelle de la Tâche de Régression :

Ce problème s'inscrit dans le cadre de l'apprentissage supervisé. Il s'agit d'une tâche de régression. L'objectif est de modéliser une fonction de prédiction capable d'estimer une variable cible continue à partir d'un vecteur de caractéristiques.

La performance du modèle est mesurée par l'écart entre la valeur réelle et la valeur prédite à l'aide d'indicateurs de performance tels que le RMSLE et le coefficient de détermination.

État de l'Art Synthétique :

Pour traiter des données tabulaires hétérogènes, trois grandes approches coexistent:

1. Les modèles linéaires: Rapides et transparents, ils constituent une référence de départ mais peinent à capturer les relations complexes non linéaires.

2. Les arbres de décision simples: Ils s'adaptent nativement aux non-linéarités mais souffrent d'une grande variance.

3. Les méthodes d'ensemble: En combinant plusieurs dizaines d'arbres, ces algorithmes permettent de réduire drastiquement le biais et la variance.

2. ÉTAPE 1: ANALYSE EXPLORATOIRE DES DONNÉES

L'analyse de notre fichier initial a permis de dégager trois conclusions fondamentales:

- Distribution de la cible: L'histogramme de la variable cible révèle une forte asymétrie positive. La majorité des observations se concentre sur des volumes faibles à modérés, avec de rares pics très élevés.
- Corrélations environnementales: La matrice de corrélation montre que la température réelle et ressentie possède l'impact positif le plus fort sur la demande.
- Profils horaires: L'analyse graphique de la demande par heure met en évidence un comportement bimodal très marqué.

3. ÉTAPE 2: PRÉ-PROCESSING ET FEATURE ENGINEERING

Choix Méthodologiques :

Le jeu de données de test contient les colonnes utilisateurs occasionnels et abonnés. Comme la variable cible est la somme exacte de ces deux colonnes (**casual** et **registered**), inclure ces variables dans notre espace d'entraînement permettrait à n'importe quel modèle linéaire de trouver une solution parfaite de manière triviale.

Afin d'éviter toute fuite de données (**data leakage**), les variables **casual** et **registered** ont été exclues du modèle. La prédiction repose uniquement sur les variables météorologiques et temporelles.

Pipeline de Transformation des Données :

Pour garantir l'étanchéité des traitements, l'intégralité du pré-processing a été codée à l'aide d'objets standardisés de Scikit-Learn :

- Transformateur Temporel Personnalisé: La chaîne de caractères brute a été convertie pour extraire l'heure, le mois, le jour de la semaine et l'année.
- Traitement des variables catégorielles: Les variables temporelles extraites ainsi que la météo et les saisons ont été encodées via un **OneHotEncoder**.
- Traitement des variables numériques : Les variables continues ont été traitées par un **StandardScaler** pour mettre toutes les caractéristiques sur une échelle identique.

4. ÉTAPE 3 : ÉVALUATION DE BASE

Afin d'identifier scientifiquement les meilleurs algorithmes pour notre problème, nous avons mis en place un banc d'essai évaluant **10 modèles distincts** sans aucun réglage d'hyperparamètres.

Évaluation :

Chaque modèle a été soumis à une **validation croisée à 5 plis**. Trois indicateurs ont été calculés simultanément : le RMSLE, la MAE et le coefficient de détermination.

Analyse des Résultats et Graphiques :

Les résultats graphiques ont révélé une hiérarchie indiscutable :

- Les Modèles Linéaires: Ils échouent tous avec un RMSLE très élevé.
- Le Modèle de Proximité: Mauvaise performance.
- Le Trio de Tête (Méthodes d'ensemble): Les algorithmes basés sur des arbres de décision s'imposent largement.

--- TABLEAU FINAL DES RÉSULTATS MULTICRITÈRES ---

Modèle	RMSLE Moyen	RMSLE Std	MAE Moyenne	R ² Moyen
Hist Gradient Boosting	0.3518	0.0046	0.2464	0.9375
Extra Trees	0.3927	0.0093	0.2645	0.9222
Random Forest	0.4310	0.0048	0.2950	0.9062
Gradient Boosting	0.5376	0.0089	0.3997	0.8541
Linear Regression	0.5912	0.0069	0.4423	0.8236
Ridge Regression	0.5912	0.0069	0.4427	0.8236
Decision Tree	0.5921	0.0137	0.3859	0.8231
K-Neighbors Regressor	0.8326	0.0147	0.5958	0.6503
ElasticNet Regression	1.3937	0.0151	1.1312	0.0202
Lasso Regression	1.4085	0.0151	1.1450	-0.0007

5. ÉTAPE 4 : TUNING, OPTIMISATION ET SELECTION DU MODÈLE FINAL

Stratégie d'Optimisation :

Les trois modèles retenus ont fait l'objet d'une recherche d'hyper paramètres afin de maximiser leur précision :

- Une recherche exhaustive par grille a été appliquée sur le **Hist Gradient Boosting**.
- Une recherche aléatoire contrôlée a été appliquée sur les modèles de forêts.

Le modèle ayant montré la progression la plus solide et l'erreur la plus faible à l'issue de cette phase est le **Hist Gradient Boosting Regressor**.

Résultats Finaux sur le Jeu de Test Étanche :

Après avoir été entraîné sur la totalité des données d'apprentissage avec ses hyper paramètres optimaux, le modèle a été évalué sur le fichier test. Les résultats obtenus sont d'un excellent niveau :

- **RMSLE Final (Test)** : 0.4282
- **MAE Finale (Test)** : 0.3335
- **Coefficient de Détermination (R² Test)** : 0.9050

Un score de R^2 de 90,50 % confirme que notre modèle est capable de prédire avec une très haute précision le comportement des utilisateurs du réseau de vélos.

Diagnostic de l'Overfitting

L'overfitting se traduit par un modèle excellent sur ses données d'entraînement mais incapable de généraliser sur de nouvelles données. Pour valider notre approche, nous comparons notre erreur de validation interne à notre erreur de test réelle :

- **RMSLE Moyen en Validation Croisée** : 0.3698
- **RMSLE Réel sur le Jeu de Test** : 0.4282
- **Écart (Test validation)** : 0.0584

L'écart observé est très petit, inférieur à 0,06. Le modèle a bien assimilé les règles générales de la météo et du calendrier sans mémoriser les erreurs statistiques du fichier d'entraînement.

6. ÉTAPE 5 : INDUSTRIALISATION ET SÉRIALISATION DU PIPELINE

Au lieu d'exporter seulement le modèle prédictif, nous avons décidé d'encapsuler toute la chaîne de traitement dans un objet unique appelé Pipeline de Scikit-Learn. Cet objet combine notre extracteur de dates, notre encodeur/scatler et l'algorithme de boosting final.

Cet ensemble a été enregistré au format binaire dans un fichier nommé **bike_sharing_model.joblib**.

Cette méthode d'ingénierie présente un avantage important pour l'industrie : elle élimine le problème de Training-Serving Skew (différence de comportement du code entre la phase de recherche et la phase de production). Lors de la mise en production, l'application reçoit une ligne de données brutes, charge le fichier **.joblib** et appelle la méthode **.predict()**. Tout le prétraitement est effectué de manière transparente en arrière-plan.

7. ÉTAPE 6 : ARCHITECTURE DE DÉPLOIEMENT ET PRODUCTION

Choix technologique :

Pour rendre le modèle accessible aux utilisateurs ou aux équipes métiers, nous avons créé une application web interactive en utilisant le framework Streamlit, qui est déployée de manière publique sur Streamlit Cloud.

Fonctionnement de l'application (app.py) :

L'interface utilisateur est simple et facile à utiliser, avec des widgets tels que des curseurs et des menus déroulants. Une fois que les paramètres sont saisis, l'application crée un DataFrame brut, appelle le pipeline globalisé pour exécuter l'extraction et l'encodage de manière transparente, puis utilise la fonction **np.expm1()** pour restituer un nombre entier et cohérent de vélos disponibles.

Pour respecter les meilleures pratiques de production logicielle, nous avons isolé la définition de notre transformateur personnalisé **TemporalFeatureExtractor** dans un module Python indépendant nommé **model.py**. Cela nous permet de résoudre le problème de la désérialisation avec joblib. En important explicitement cette classe dans notre script principal, nous nous

assurons que l'interpréteur Python sur le serveur distant Streamlit Cloud connaît parfaitement la structure de l'objet lors de sa reconstruction en mémoire.

Déploiement sur Streamlit Cloud :

Pour rendre l'application publique les fichiers ont été envoyés sur un dépôt GitHub structuré contenant le script (app.py), le fichier binaire (bike_sharing_model.joblib), fichier README.md et un fichier de configuration des dépendances (requirements.txt). Le dépôt a ensuite été lié à la plateforme Streamlit Cloud, automatisant l'installation des packages nécessaires et fournissant une URL d'accès public et sécurisé.

8. CONCLUSION

Ce projet a permis de valider avec rigueur l'ensemble du cycle de vie d'une solution de Machine Learning, depuis la phase d'exploration des données jusqu'au déploiement dans le cloud.

L'exclusion des variables de fuite (casual et registered), combinée à un feature engineering temporel précis et à la puissance de l'algorithme **Hist Gradient Boosting Regressor**, a permis d'aboutir à un modèle très performant, capable d'expliquer plus de 90 % de la demande sur des données inconnues.

L'approche par Pipeline Scikit-Learn garantit une transition fluide et sans bug vers la production, offrant un outil d'aide à la décision fiable et directement exploitable pour la gestion opérationnelle du parc de vélos.